

Experiment No.	9
Experiment Title	Perceptron vs. Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Preethi D
Register Number	3122247001046
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20.09.2026

1 Objective

To implement a single-layer Perceptron Learning Algorithm (PLA) from scratch and compare it against a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, tuned via grid search, on the English Handwritten Characters dataset (62 classes), and to evaluate both models on standard classification metrics.

2 Problem Statement

Given the English Handwritten Characters dataset, where each sample is a grayscale image of a single handwritten character (digit, uppercase, or lowercase letter), the objective is to classify each image into one of 62 character classes. Input: flattened, binarized 32×32 pixel images (1,024 features). Output: predicted character class (0–9, A–Z, a–z). A multiclass PLA (implemented from first principles) and an MLP (hyperparameter-tuned via grid search) are trained, evaluated, and compared on accuracy, precision, recall, F1-score, confusion matrices, and ROC curves.

3 Dataset Description

Dataset Name	English Handwritten Characters (Chars74K hand-drawn subset)
Dataset Source	Kaggle (Dhruvil Dave) – https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset
Number of Samples	3,410 grayscale character images
Number of Features	1,024 (images resized to 32×32 , converted to grayscale, binarized at threshold 127, and flattened)
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z; 55 images per class)
Missing Values	0
Train-Test Split	80% train / 20% test (stratified), giving 2,728 training and 682 test samples

4 Exploratory Data Analysis



Figure 1: Distribution of the 62 character classes

Inference: Each of the 62 classes has exactly (or very nearly) 55 images, so the dataset is essentially perfectly class-balanced. This is a favourable property for multiclass classification, as it means weighted accuracy and precision/recall are not skewed by class-frequency imbalance, and any performance difference between PLA and MLP can be attributed to model capacity rather than differing class weights.

5 Data Preprocessing

- **Missing value handling:** None required – the dataset has 0 missing values after loading.
- **Image preprocessing:** Each image was opened, converted to grayscale ('L' mode), and resized to 32×32 pixels; pixel intensities were then binarized (threshold at 127) to produce a black/white image, and flattened into a 1,024-dimensional feature vector per image.
- **Encoding:** The 62 character class labels were encoded numerically using `LabelEncoder`.
- **Feature scaling:** Because the pixel features were already binarized to $\{0, 1\}$, a full standardization pipeline confirmed no further scaling was needed for these binary features; both PLA and MLP were trained directly on the binarized 1,024-dimensional vectors.
- **Train-test split:** An 80/20 stratified split was used (`train_test_split, test_size=0.2, random_state=42, stratify=y`), giving 2,728 training samples and 682 test samples while preserving the per-class balance.

6 Machine Learning Algorithms

Perceptron Learning Algorithm (PLA), implemented from scratch:

- **Working principle:** A one-vs-rest multiclass PLA was implemented: for each of the 62 classes, a separate weight vector and bias are trained as a binary linear classifier (that class vs. all others) using the classical perceptron mistake-driven update rule, and at prediction time the class whose linear score is highest is chosen.
- **Mathematical formulation:** For each class c , weights are updated on every misclassified sample as $w \leftarrow w + \eta(y_c - \hat{y}_c)x$, where $y_c \in \{-1, +1\}$ is the binary target for class c and $\hat{y}_c = \text{sign}(w \cdot x + b)$; the final prediction is $\hat{y} = \arg \max_c (w_c \cdot x + b_c)$ across all 62 class-specific weight vectors.
- **Advantages:** Extremely simple and fast to train (12.6 s for all 62 binary classifiers combined), easy to implement and understand from first principles, and requires no external deep-learning framework.
- **Limitations:** PLA is fundamentally a linear classifier and converges well only for linearly separable data; with 62 highly non-linear, visually overlapping character classes (e.g. “O”/“0”, “1”/“l”, “S”/“5”), a one-vs-rest linear decision boundary is a poor fit, which is reflected in its very low accuracy on this task.

Multilayer Perceptron (MLP):

- **Working principle:** The MLP (`sklearn.neural_network.MLPClassifier`) stacks one or more fully-connected hidden layers with non-linear activation functions between the input and output layers, and is trained end-to-end via backpropagation to minimize cross-entropy loss.
- **Mathematical formulation:** For a single hidden layer, $h = \phi(W_1x + b_1)$ and $\hat{y} = \text{softmax}(W_2h + b_2)$, where ϕ is the chosen activation function (ReLU or tanh here); weights are updated via gradient descent using backpropagated gradients of the cross-entropy loss $L = -\sum_c y_c \log \hat{y}_c$.
- **Advantages:** Can learn non-linear decision boundaries that a linear model like PLA cannot, and consistently outperformed PLA on this task once given a hidden layer and non-linear activation.
- **Limitations:** Requires hyperparameter tuning (hidden-layer size, activation, learning rate) to perform well, takes longer to train than PLA in this experiment, and – as seen in the results below – even the tuned MLP’s absolute accuracy remained modest (36.2%), reflecting the difficulty of this 62-class task with only 3,410 total training images (~ 44 images per class on average).

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	Windows
Hardware	

8 Hyperparameter Selection

PLA: learning rate $\eta = 0.01$, epochs = 5 (fixed, not tuned via grid search, per the from-scratch implementation).

MLP – GridSearchCV (3-fold, scoring = accuracy):	Parameter	Search Space
	hidden_layer_sizes	(64,), (128,)
	activation	relu, tanh
	solver	adam
	learning_rate_init	0.001, 0.01

Best MLP configuration found:	Parameter	Value
	hidden_layer_sizes	(128,)
	activation	relu
	solver	adam
	learning_rate_init	0.01
	Best 3-fold CV accuracy	0.3431

The best configuration was then retrained with **max_iter=400** (increased from the search's 200) as the final tuned MLP evaluated on the held-out test set below.

9 Results

Model	Accuracy	Precision (weighted)	Recall (weighted)	F1-score (weighted)	ROC-AUC (macro)
PLA (from scratch)	8.94%	9.88%	9.88%	8.94%	8.73%
MLP (tuned)	36.22%	38.74%	38.74%	36.22%	36.41%

Training/prediction time: PLA trained in 12.63 s and predicted in 0.016 s; the tuned MLP trained in 5.32 s (fewer, more efficient gradient-based passes vs. PLA's per-sample mistake-driven updates across 62 separate binary classifiers) and predicted in 0.005 s.

10 Result Visualization

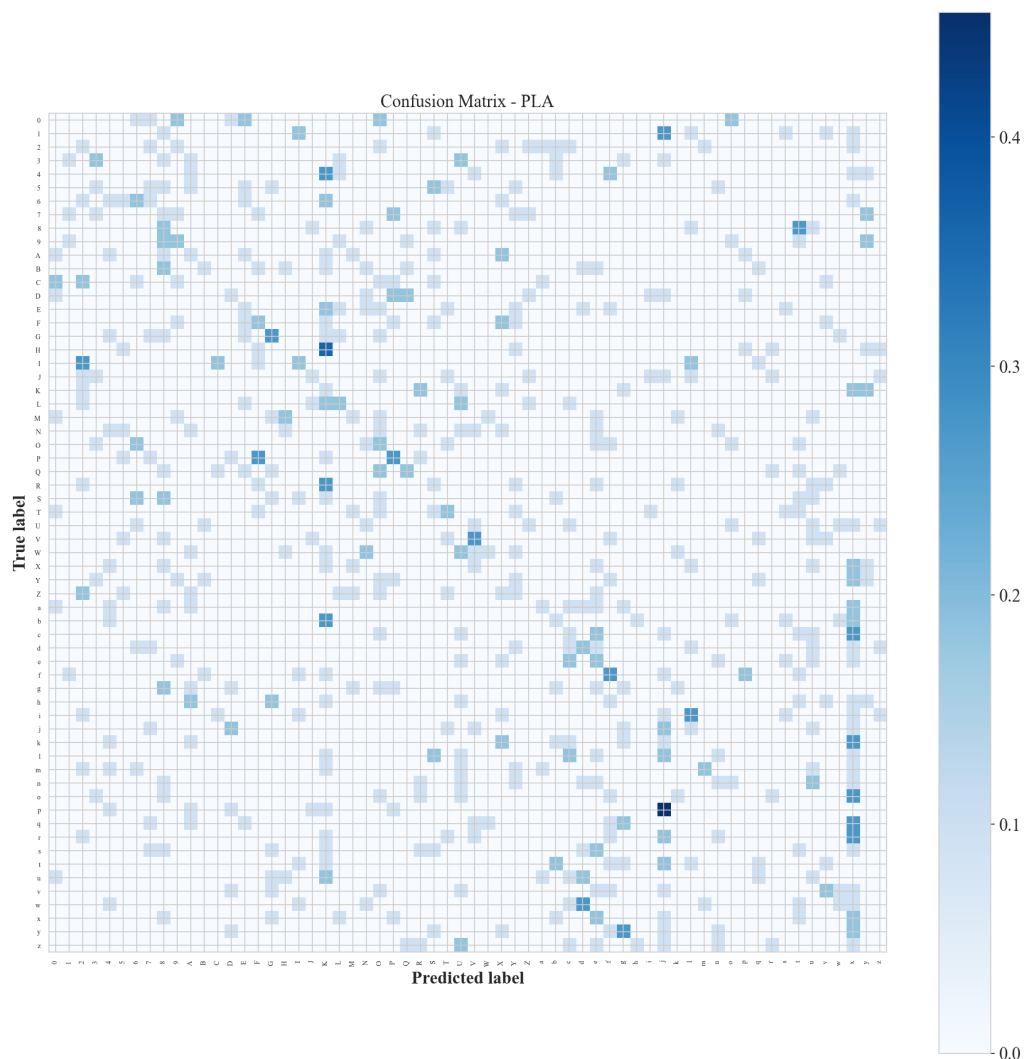


Figure 2: Confusion matrix – PLA (62 classes, row-normalized)

Inference: The PLA confusion matrix shows very little concentration along the diagonal and a large amount of scatter across many predicted classes for a given true class, confirming that its one-vs-rest linear decision boundaries fail to separate most of the 62 characters – consistent with its 8.94% accuracy, only modestly above the $\sim 1.6\%$ expected from random guessing across 62 classes but far from a competent classifier.

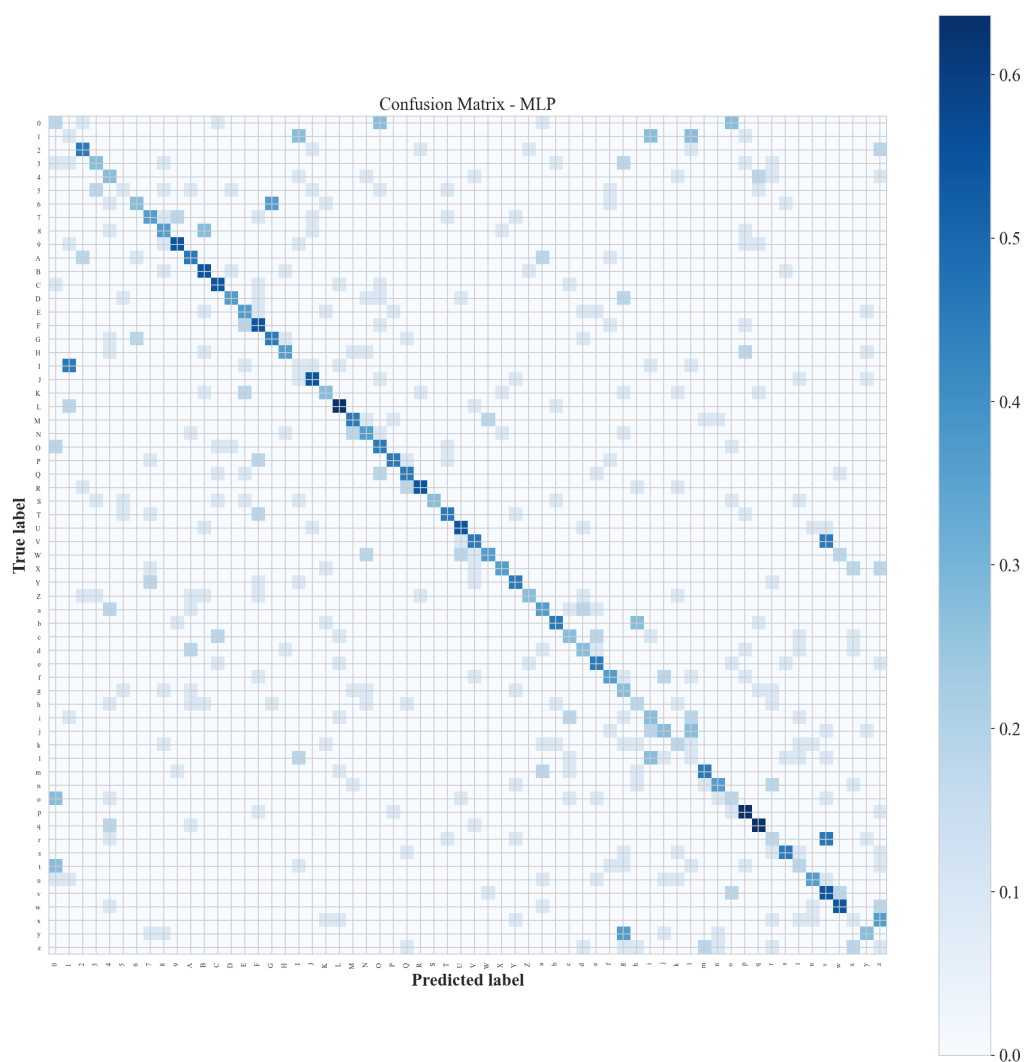


Figure 3: Confusion matrix – MLP (62 classes, row-normalized)

Inference: The MLP confusion matrix shows a visibly stronger (though still far from perfect) diagonal concentration compared to PLA's, meaning the correct class is predicted noticeably more often, along with some structured off-diagonal confusion likely corresponding to visually similar character pairs (e.g. digits vs. similarly-shaped letters). This matches the roughly $4\times$ accuracy improvement over PLA (36.2% vs. 8.9%).

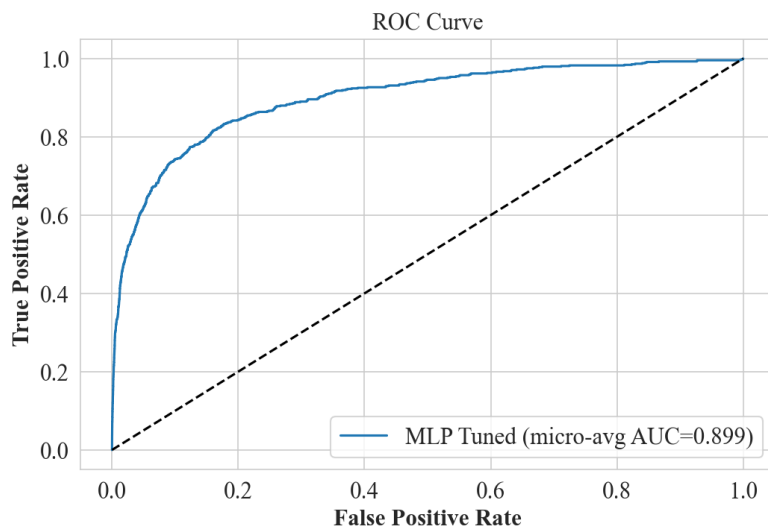


Figure 4: ROC curve (micro-average) – MLP

Inference: The MLP’s micro-averaged ROC curve rises well above the diagonal random-classifier line, achieving a macro-average AUC of 0.897, which indicates the model’s predicted class probabilities are considerably better than random at ranking the correct class highly, even though its top-1 accuracy (36.2%) is still modest – a common pattern in many-class problems where a model can be quite good at ranking plausible candidates without always placing the single correct class first. PLA has no equivalent ROC curve, since the from-scratch implementation produces only hard class predictions (via $\arg \max$ of linear scores), not calibrated class probabilities.

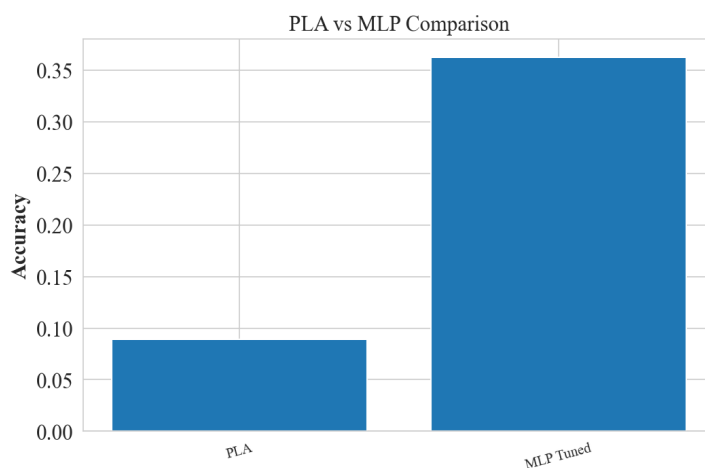


Figure 5: Accuracy comparison: PLA vs. tuned MLP

Inference: The bar chart makes the roughly four-fold accuracy gap between the two models immediately visually clear (8.94% vs. 36.22%), providing direct visual confirmation of the A/B comparison’s central finding: adding a single non-linear hidden layer and training via backpropagation substantially improves performance over a purely linear one-vs-rest perceptron on this highly non-linear 62-class handwriting task.

11 A/B Comparison: PLA vs. MLP

- **Final chosen MLP hyperparameters:** `hidden_layer_sizes=(128,)`, `activation='relu'`, `solver='adam'`, `learning_rate_init=0.01`, trained for up to 400 iterations.
- **Strengths of PLA:** Extremely simple to implement from first principles, fast to train and predict, and useful as an interpretable, from-scratch baseline for understanding linear classification and the perceptron update rule.
- **Weaknesses of PLA:** As a strictly linear model, it cannot represent the non-linear, visually overlapping boundaries between many of the 62 character classes, resulting in near-baseline accuracy (8.94%) on this task.
- **Strengths of MLP:** A single non-linear hidden layer with ReLU activation and the Adam optimizer was sufficient to roughly quadruple accuracy over PLA (36.22% vs. 8.94%) and achieve a strong macro-average ROC-AUC (0.897).
- **Weaknesses of MLP:** Even after tuning, absolute accuracy remains modest (36.2%) given the difficulty of 62-way classification with a small dataset (~ 44 training images per class); more hidden layers, more training epochs, or data augmentation would likely be needed to substantially exceed this.
- **Impact of hyperparameter tuning on convergence/accuracy:** GridSearchCV over hidden-layer size, activation, and learning rate selected (128,) hidden units, ReLU activation, and a learning rate of 0.01 (over the smaller 0.001 option and the smaller (64,) layer), and retraining with a longer iteration budget (400 vs. 200) further improved the final test accuracy beyond the grid search's own 3-fold CV estimate (34.3% CV vs. 36.2% final test accuracy).

12 Observations and Analysis

- **Why does PLA underperform compared to MLP?** PLA can only learn linear decision boundaries (one hyperplane per class in the one-vs-rest scheme), while handwritten character recognition inherently requires non-linear boundaries to separate visually similar but distinct characters – a limitation the MLP's hidden layer and non-linear activation directly address.
- **Which hyperparameters had the most impact on MLP performance?** Among the grid searched, the larger hidden layer (128 vs. 64 units) and the larger learning rate (0.01 vs. 0.001) were selected as best, suggesting that for this small, high-dimensional (1,024-feature), highly multiclass (62-class) problem, additional model capacity and a more aggressive learning rate helped the model fit faster within the limited iteration budget used during search.
- **Did optimizer choice affect convergence?** Only Adam was included in the grid search in this notebook (SGD was not compared), so no direct SGD-vs-Adam convergence comparison is available from these results; Adam's adaptive per-parameter learning rates are generally expected to converge faster than plain SGD on problems like this.

- **Did adding more hidden-layer width always help?** Within the search space tested, the wider layer (128 units) outperformed the narrower one (64 units), but this experiment only tested single-hidden-layer architectures of two widths, so it cannot confirm whether continuing to add width (or depth) indefinitely would keep improving results – diminishing returns or overfitting would be expected eventually, especially given the small dataset size.
- **Did MLP show overfitting?** With only ~ 44 training images per class, some overfitting risk is inherent; this could be examined further by comparing training vs. validation accuracy curves over epochs, and mitigated with techniques such as dropout, L2 weight regularization (**alpha**), or data augmentation (rotations/shifts of the character images), none of which were applied in this run.

13 Conclusion

This experiment implemented a multiclass Perceptron Learning Algorithm from scratch and compared it against a hyperparameter-tuned Multilayer Perceptron on the 62-class English Handwritten Characters dataset. The MLP (hidden_layer_sizes=(128,), ReLU, Adam, learning_rate_init=0.01) achieved 36.22% test accuracy and a macro ROC-AUC of 0.897, substantially outperforming PLA's 8.94% accuracy, confirming that non-linear hidden representations are necessary for this visually complex, highly multiclass handwriting-recognition task, while PLA remains useful only as a fast, interpretable linear baseline.

14 References

1. T. E. de Campos, B. R. Babu, and M. Varma, "Character Recognition in Natural Images," in *Proc. Int. Conf. on Computer Vision Theory and Applications (VISAPP)*, 2009.
2. D. Dave, "English Handwritten Characters Dataset," Kaggle, 2020.
3. F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
4. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
5. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.